In [1]:
```python
# Task 1: General class for the tree object
class TreeNode:
    def __init__(self, weight):
        self.weight = weight
        self.children = []

# Task 2: Generate tree of depth N, with root weight 1/n
def build_fb_tree(n, depth):
    # To ensure the total sum of the tree is 1,
    # the root weight must be 1/depth (since each level's sum equals the root's weight).
    # Based on the prompt "returns 1 for various n's", we set depth = n.
    root_weight = 1.0 / depth
    root = TreeNode(root_weight)

    current_level = [root]
    # Build the tree level by level (depth-1 remaining levels)
    for _ in range(depth - 1):
        next_level = []
        for node in current_level:
            for _ in range(n):
                # Each child is 1/n of its parent's weight
                child = TreeNode(node.weight / n)
                node.children.append(child)
                next_level.append(child)
        current_level = next_level

    return root
```

In [2]:
```python
from collections import deque

# Task 3: Depth First Recursive Function
def dfs_sum(node):
    if not node:
        return 0
    total = node.weight
    for child in node.children:
        total += dfs_sum(child)
    return total
```

```python
# Task 4: Breadth First Non-recursive Function
def bfs_sum(root):
    if not root:
        return 0
    total = 0
    queue = deque([root])
    while queue:
        node = queue.popleft()
        total += node.weight
        for child in node.children:
            queue.append(child)
    return total

# Test for various n's
for n in [3, 4, 5]:
    tree_root = build_fb_tree(n=n, depth=n)
    print(f"--- Test with n={n} ---")
    print(f"DFS Sum: {round(dfs_sum(tree_root), 5)}")
    print(f"BFS Sum: {round(bfs_sum(tree_root), 5)}\n")
```

```
--- Test with n=3 ---
DFS Sum: 1.0
BFS Sum: 1.0

--- Test with n=4 ---
DFS Sum: 1.0
BFS Sum: 1.0

--- Test with n=5 ---
DFS Sum: 1.0
BFS Sum: 1.0
```

In [3]:

```python
# Task 5: Search and flip sign
def dfs_sum_and_flip(node):
    if not node: return 0
    total = node.weight
    node.weight *= -1  # Flip sign AFTER reading the value
    for child in node.children:
        total += dfs_sum_and_flip(child)
    return total
```

```python
def bfs_sum_and_flip(root):
    if not root: return 0
    total = 0
    queue = deque([root])
    while queue:
        node = queue.popleft()
        total += node.weight
        node.weight *= -1  # Flip sign AFTER reading the value
        for child in node.children:
            queue.append(child)
    return total

# Test Task 5
print("--- Task 5: Sign Flipping Test (n=3) ---")
root_flip = build_fb_tree(n=3, depth=3)
print("1st search (DFS) returns:", round(dfs_sum_and_flip(root_flip), 5))
print("2nd search (BFS) returns:", round(bfs_sum_and_flip(root_flip), 5))


# Task 6: Recursive version of Breadth First
def bfs_recursive(queue):
    if not queue:
        return 0
    node = queue.popleft()
    total = node.weight
    for child in node.children:
        queue.append(child)
    # The recursion happens here, passing the updated queue state
    return total + bfs_recursive(queue)

print("\n--- Task 6: Recursive BFS Test ---")
root_rec = build_fb_tree(n=3, depth=3)
q = deque([root_rec])
print("Recursive BFS Sum:", round(bfs_recursive(q), 5))
```

```
--- Task 5: Sign Flipping Test (n=3) ---
1st search (DFS) returns: 1.0
2nd search (BFS) returns: -1.0

--- Task 6: Recursive BFS Test ---
Recursive BFS Sum: 1.0
```

Task 7: Why recursive BFS is NOT recommended?

While Depth First Search (DFS) elegantly matches the call stack due to its LIFO (Last-In-First-Out) nature, implementing Breadth First Search (BFS) recursively is considered an anti-pattern for two major reasons related to system memory:

1. **Stack Overflow Risk (Space Complexity):** In a recursive DFS, the maximum recursion depth is bounded by the height of the tree $O(N)$. However, a recursive BFS requires maintaining the queue state across function calls. Because it visits nodes level by level, the recursion depth equals the total number of nodes in the tree $O(n^N)$. For a tree with just $n = 10$ and depth $N = 5$, this forces $100,000$ consecutive frames onto the call stack, which will immediately trigger a (or in Python). `StackOverflowError` `RecursionError`
2. **Memory Overhead:** Even if the stack limit is raised, each recursive call allocates a new stack frame (storing local variables, return addresses, etc.). Iterative BFS using a standard Heap-allocated Queue is vastly more memory-efficient and avoids the massive overhead of function call context switching.

Task 8: GitHub Deployment

Note: The Jupyter Notebook `Problem_Set_6.ipynb` containing this discussion and code implementation has been committed and pushed to the repository in time.

In [ ]: